HypG: A Post-Quantum Cryptographic Suite in Safe Rust

White Paper — Design, Architecture, and Evaluation (v2.0)

HypG Project

October 2026

Abstract

HypG is a from-scratch post-quantum cryptographic suite written in 100% safe Rust with zero dependencies: a Module-LWE key-encapsulation mechanism, BLAKE3-based hashing, ChaCha20 / keyed-BLAKE3 authenticated encryption (one-shot and streaming), and stateless hash-based signatures. The suite is a *hardened research profile* (v2.0): parameters now match NIST Category-5 dimensions, every construction is implemented, deterministically tested, adversarially exercised, and fully gated in CI — but unaudited, with unconfirmed bit-security, and explicitly **not for deployment**. Matching NIST dimensions is not a security certification. This paper describes the design rationale, the architecture, the evaluation results, and the honest limits of what a local machine can prove.

Contents

1 Introduction **2**
1.1 What HypG is not . . . 2

2 Design principles **2**

3 Architecture **2**
3.1 Parameters . . . 3

4 Pillar designs **3**
4.1 KEM: Module-LWE with Fujisaki–Okamoto . . . 3
4.2 Hash: BLAKE3, nothing else . . . 3
4.3 AEAD: Encrypt-then-MAC, one-shot and streaming . . . 3
4.4 Signatures: SLH-shaped hypertree . . . 4

5 Implementation posture **4**

6 Evaluation **4**
6.1 Correctness gates . . . 4
6.2 Adversarial testing (S1–S14) . . . 4
6.3 Performance (Intel i5-14400F, release) . . . 4

7 Limits and threat model **5**

8 Road to a security claim **5**

9 Conclusion **5**

1 Introduction

The arrival of cryptographically relevant quantum computers would break the public-key cryptography in wide use today (RSA, Diffie–Hellman, elliptic curves). The post-quantum response centers on new hardness assumptions — structured lattices chief among them — and on conservative symmetric cryptography, which survives largely unscathed.

HypG is an attempt to build a *complete* post-quantum suite, not just one primitive: key establishment, hashing, authenticated encryption, and signatures, sharing a single implementation core. It is written from scratch in safe Rust, with no dependencies and no `unsafe` code, targeting `no_std` environments for the cryptographic libraries. Version 2.0 hardens the parameter profile as a research prototype: functionally complete and obsessively tested, but unaudited and making no security claim beyond its explicitly stated position.

1.1 What HypG is not

To avoid misunderstanding: HypG is not a standard, not a submission to any standardization process, not audited, and not deployable. Its KEM targets NIST Category-5 dimensions with *unconfirmed* bit-security. Its test vectors pin behavior, not security. Section 7 documents these limits in full; the freeze certifies *process completeness* — everything checkable on a local machine is checked — not security.

2 Design principles

Five principles survived every design review and are enforced mechanically in continuous integration:

1. **Standard primitives, used as designed.** BLAKE3 for hashing, ChaCha20 for encryption, Module-LWE for the KEM trapdoor, SLH-DSA-shaped hash-based signatures. No novel symmetric constructions: proposals for a custom sponge, a novel keystream permutation, and truncated 16-byte tags were evaluated and *rejected*, because each would trade oracle-validated assurance for unanalyzed novelty.

2. **Zero attack surface from the supply chain.** Zero dependencies — `cargo tree` shows workspace crates only, and `cargo audit` is clean. There is nothing upstream to compromise.

3. **Memory safety by construction.** `#![forbid(unsafe_code)]` in every library, gated by CI. Miri (Rust's undefined-behavior interpreter) runs clean over the v2.0 core and symmetric crates; the KEM re-run is pending.

4. **Determinism and bit-exactness.** Same inputs produce same outputs on x86-64 and AArch64, in debug and release builds. Little-endian serialization is explicit everywhere (a CI gate forbids endian-dependent conversions), and outputs are pinned by frozen known-answer test files.

5. **Fail closed.** KEM decapsulation never errors — forgeries yield a pseudorandom secret via implicit rejection. AEAD authentication failures return `None` and erase tentative plaintext. The operator CLI distinguishes usage errors (exit 2) from authentication failures (exit 3).

3 Architecture

A single `no_std` core (`hypg-core`) provides the cellular-automaton expander (`CA-PRF`), Module-LWE polynomial arithmetic, portable BLAKE3, ChaCha20, and constant-time helpers. Four pillars build on it:

Pillar	Crate(s)	Construction
KEM	`hypg-kem`	Module-LWE + Fujisaki–Okamoto, implicit rejection
Hash	`hypg-hash`	BLAKE3 API: 32/64-byte digests, XOF, keyed, KDF
AEAD	`hypg-symm`, `hypg-aead`	ChaCha20 + keyed-BLAKE3 EtM, one-shot and streaming
Signatures	`hypg-ds`, `hypg-mt`	SLH-DSA-256f-shaped FORS + 17-layer XMSS hypertree

Five std harnesses surround the libraries (all zero-dependency): `hypg-kat` (deterministic vector printer + frozen files), `hypg-bench` (throughput + timing harness), `hypg-redteam` (14 adversarial suites), `hypg-lattice` (local lattice lab), and `hypg-cli` (operator tool). Threading lives only in `hypg-mt` (a scoped-thread signer producing byte-identical output) — never in the `no_std` libraries, because threads are a platform capability, not a primitive.

3.1 Parameters

KEM	$n = 256$, $k = 4$ (dimension 1024), $q = 2^{12}$, CBD $\eta = 2$, 1 bit/coefficient $pk = 1568$ B, $ct = 1440$ B, $sk = 1632$ B, $ss = 32$ B
Hash	32/64-byte digests, arbitrary-length XOF
AEAD	32 B key, 12 B nonce, 32 B tag, $ct.len() = pt.len()$, stream $< 2^{38}$ B
Signatures	256f shape: $pk = 64$ B, $sk = 128$ B, $sig = 49992$ B

The power-of-two modulus $q = 4096$ (rather than Kyber's 3329) is a documented deviation enabling branchless mask-only reduction. Version 2.0 is a breaking wire-format change: the KEM module rank doubles and the signature profile moves to the 256f shape. v1.0 vectors remain in the repository as historical evidence.

4 Pillar designs

4.1 KEM: Module-LWE with Fujisaki–Okamoto

`KeyGen` samples secret and error vectors from a caller-supplied seed via the `CA-PRF` expander, builds $b = A \cdot s + e$ over $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$, and outputs $pk = seed_A \,\|\, \mathrm{pack}_{12}(b)$. `Encaps` derives encryption randomness deterministically from the message and public key (the FO G oracle), and the shared secret binds message, ciphertext, and public-key hash. `Decaps` re-encrypts and selects between the real and a fake secret in constant time (implicit rejection). Decryption noise margin is analytic: ± 1024 against ~ 60 standard deviations, confirmed by 20000/20000 clean round-trips at maximum load.

4.2 Hash: BLAKE3, nothing else

The hashing pillar is a thin ergonomic API over portable BLAKE3 — 32-byte hashes, 64-byte XOF digests (whose first 32 bytes equal the hash, by construction), keyed hashing, key derivation, expansion, and a streaming hasher proven equivalent to one-shot across split patterns. An earlier from-scratch Keccak was retired in favor of the standard primitive with official test vectors, cross-checked against third-party bindings at freeze time.

4.3 AEAD: Encrypt-then-MAC, one-shot and streaming

Encryption is IETF ChaCha20; authentication is keyed BLAKE3 over $nonce \,\|\, ad \,\|\, ct \,\|\, \mathrm{le64} \,\|\, \mathrm{le64}$ with per-operation subkeys derived via BLAKE3 `derive_key`. The streaming front uses the *identical* byte schedule (same subkeys, same MAC input order, same counter sequence with cross-chunk partial-block buffering), so any chunking yields the same bytes as one shot — proven by

equivalence tests across adversarial chunkings, which is what lets the existing KAT vectors cover the streaming format with no new frozen files. Notably, these tests caught a genuine keystream-skipping bug during development (counter restarted per chunk), fixed with cross-chunk block buffering.

4.4 Signatures: SLH-shaped hypertree

FORS (35 trees of 512 leaves) signs the message digest; a 17-layer XMSS hypertree of WOTS+ chains ($w = 16$) authenticates the FORS key; hedged randomness keeps signing deterministic. Every WOTS+ chain runs all 15 steps with branchless selection (constant-time rule), and reserved padding in the 49992-byte signature is rejection-checked at verification. A threaded signer parallelizes the 52 independent tree builds for a $\sim$3.7$\times$ speedup with byte-identical output.

5 Implementation posture

`no_std` libraries contain no allocation, no floats, no OS randomness, and no panicking constructs (CI bans `panic!`, `unwrap`, and `expect` even in tests). Callers supply all randomness; all loops have public bounds; secret-dependent branches and indices are prohibited by policy and reviewed by inspection. Key material gets best-effort erasure (compiler-fenced overwrites; true volatile erasure would require `unsafe`, which policy forbids), and debug renderings of key-holding types are redacted.

6 Evaluation

6.1 Correctness gates

129 tests pass on stable (debug and release) and on the MSRV 1.70 toolchain: 96 unit, 20 CLI failure-path integration, 13 doc-tests. Four frozen v2.0.0 KAT files pin behavior byte-for-byte; debug and release outputs are identical. Cross-architecture v2.0 re-diffing passed: a musl-static KAT binary under `qemu-aarch64` matched x86-64 byte-for-byte (234114/234114 bytes). Miri is clean for the v2.0 core and symmetric crates; the KEM re-run is pending.

6.2 Adversarial testing (S1–S14)

The red-team harness runs 14 suites: garbage robustness, malleability sweeps (all 11520 ciphertext bits flip the secret), an 8×8 key-mismatch matrix, message extremes, FO binding, sampler statistics (524288 CBD coefficients, mean exactly 0), a 20000-trial failure sweep (zero failures), determinism, AEAD forgery sweeps, signature forgery sweeps, a mega-suite (burst faults, tampered keys, garbage matrices, pad flips, multi-sign, 1 MiB AEAD), a 1011-case panic-freedom corpus, secret-key bit flips, and streaming chunk games. All pass, including at maximum load in release builds.

6.3 Performance (Intel i5-14400F, release)

Hash / streaming / XOF (1 MiB)	818 / 820 / 807 MB/s
ChaCha20 keystream	754 MB/s
AEAD seal / open (1 MiB)	397 / 405 MB/s
KEM keygen / encaps / decaps	751 / 914 / 1107 μs
Sign / verify	45.7 ms / 2.25 ms (threaded sign: 12.6 ms)

Interleaved valid/tampered medians are recorded as per-mille smoke tests (KEM 1002pm, AEAD 581pm, DS verify 998pm); informational only, not a proof.

7 Limits and threat model

The following are explicitly *not* claimed: confirmed bit-security (the lattice-estimator run is the top backlog item), side-channel immunity beyond a branchless timing posture (no power/EM/fault analysis), reduction of the `CA-PRF` heuristic (uniformity rests on the outer BLAKE3 in the random-oracle model), or any third-party review. KAT vectors pin behavior, not security. Abuse cases — production deployment, parameter shrinking, added dependencies, invented symmetric modes — are blocked by documentation, frozen vectors, and CI gates respectively.

8 Road to a security claim

Four external milestones stand between v2.0 and any security claim: (1) lattice-estimator confirmation with parameter review ($q = 4096$ vs. 3329); (2) third-party cryptanalysis of the trapdoor, the `CA-PRF`, the EtM binding including streaming, and the DS composition; (3) timing/fault evaluation; (4) independent code audit. None is achievable on the development machine, by construction of the evidence classes involved.

9 Conclusion

HypG v2.0 delivers a complete, hardened, obsessively tested post-quantum suite in safe Rust — and an honest account of what testing can and cannot prove. The code, vectors, measurements, and gates are public and reproducible; the remaining work belongs to estimators, laboratories, and independent reviewers.

License: GNU General Public License v3.0 or later. Companion formal specification: the HypG Yellow Paper.